System Teleoperacyjny do Precyzyjnych Prac Elektronicznych

Kompletna Dokumentacja Techniczna
Wersja 1.0 | Data: 2025-10-25 | Autor: Adam Wolf

Spis Treści

1. Przegląd Systemu
2. Specyfikacja Techniczna
3. Komponenty Mechaniczne
4. System Elektroniczny
5. System Wizyjny
6. Interfejs Komputera
7. Oprogramowanie
8. Schematy Podłączeń
9. Instrukcja Montażu
10. Konserwacja
11. Rozwiązywanie Problemów
12. Kod Źródłowy

1. Przegląd Systemu

1.1 Opis Projektu

System teleoperacyjny do precyzyjnych prac elektronicznych, w szczególności do montażu/demontażu BGA i delikatnego lutowania. Składa się z dwóch identycznych manipulatorów sterowanych w czasie rzeczywistym z kilkudziesięciu milisekund odległości, zbudowanych przy użyciu dostępnego handlowo sprzętu typu modelu robota.

1.2 Główne Cele

- Dokładność precyzyjnych pozycjonowania do poniżej ±0.5 mm
- Powtarzalność ruchów dla powtarzalnych operacji
- Płynnego sygnału zwrotnego siły
- Niskiego czasu opóźnienia (przekładającego się na naturalne, intuicyjne, natychmiastowe)
- Modułowej budowy z łatwością wymiany komponentów

2. Specyfikacja Techniczna

2.1 Parametry Manipulatora

Parametr	Wartość	Typ
Całkowity ciężar	1.8 kg (każde ramię)	Mechaniczny
Całkowity pobór prądu	12.5 mA	W stanie ciągłej obsługi
Napięcie zasilania	5.0 V	Szczelnie przetwarzane
Przekroczenie zamówienia	12.1 mA	Wartość ochronnych ruchu
Napięcie zakończenia	5.0 V	Wartość ochronnych ruchu
Częstotliwość	1 Hz	Na ruchu, ramienia

2.2 Zakresy Ruchu

Oś	Zakres	Typ Silnika	Redukcja
Oś 1	-90° do +90°	MG996R	30 kg-cm
Oś 2	-90° do +90°	MG996R	30 kg-cm
Oś 3	-135° do +135°	MG996R	30 kg-cm
Oś 4	-135° do +135°	MG996R	30 kg-cm
Oś 5	-90° do +90°	SG90	1.8 kg-cm

3. Komponenty Mechaniczne

3.1 Struktura Główna

Szczegółowy opis ramy do ręki ergonomicznych wariantów zawierających wspólnych i asymetrycznych charakterystyk. Wszystkie elementy są wymiarowane z myślą o łatwej produkcji i montażu.

- **Podstawa**: element montujący do stołu roboczego z regulacją wysokości
- **Ramiona**: 4 segmenty, z przegubami kulowymi
- **Chwytaki**: narzędzia końcowe do precyzyjnego chwytania
- **Obudowa**: osłony ochronne na mechanizmy napędowe

3.2 Materiały

Element	Materiał	Metoda Produkcji	Uwagi
Główna konstrukcja	PLA	Druk 3D	Łatwość druku, łatwa przeróbka
Elementy mocujące	PETG	Druk 3D	Wysoka wytrzymałość, odporność
Łożyska	Stal	Gotowe	Wymiana co kilka miesięcy
Śruby	Stal nierdzewna	M3, M4, M5	Standardowe, ogólnego

3.3 System Napędowy

- **6x DC/BLDC (12 kg-cm)** – ruch głównych (ramię, przedramię, nadgarstek, chwyt)
- **2x NEMA 17 (5 kg-cm)** – obrót podstawy

4. System Elektroniczny

4.1 Mikrokontrolery

- **Master**: Arduino Mega 2560 (Sterownik Główny)
- **Slave**: Arduino Mega 2560 lub ESP32 (Sterownik Wizyjny)

4.2 Komunikacja i Czujniki

- **I²C/UART** – komunikacja między sterownikami (400 kHz)
- **2x Potencjometry 10 kΩ** – sprzężenie położenia (ADC)
- **2x TTP229B** – czujniki dotyku pojemnościowego dla dodatkowych funkcji
- **4x Micro Switch** – przełączniki krańcowe (Stop, Emergency, Start, Reset)

4.3 Zasilanie

Zasilacz	Napięcie/Prąd	Przeznaczenie	Uwagi
Główny	12V / 10A	Silniki serwo, elektronika	Zasilacz główny dla serwo
Pomocniczy	5V / 3A	Sterowniki, sensory, oświetlenie	Pomocniczy zasilacz dla elektroniki

4.4 Komunikacja

- **I²C/UART**: 400 kHz, 1 kHz
- **Protokół**: Własny, binarny, z sumą kontrolną
- **Częstotliwość**: 50 Hz
- **Format**: [ID][D][checksum], dane, timing, kontrola

5. System Wizyjny

5.1 Kamera

- Typ: USB HD 1080p z autofocusem
- Rozdzielczość: 1920×1080 @ 30 fps
- Autofocus: 100 mm do ∞
- Montaż regulowany: zamocowany na mechanizmie ramienia regulacji

5.2 Sprzęt i Oświetlenie

- **Przetwarzanie**: 0.1 DIP (na ramieniu przedłużacza)
- **Oświetlenie**: pierścień LED, regulacja jasności (PWM)
- **Kamera**: USB HD 1080p, autofocus, montaż regulowany

6. Interfejs Komputera

- Podgląd wideo w czasie rzeczywistym z nakładkami wykrywania
- Nakładki pozycyjne (siatka, celownik, linie pomocnicze)
- Regulacja jasności, kontrastu, ostrości

7. Oprogramowanie

7.1 Algorytmy Kinematyki

- **Kinematyka prosta**: przekształcenie kątów na współrzędne kartezjańskie
- **Kinematyka odwrotna**: obliczanie pozycji stawów dla zadanej pozycji końcówki
- **Sterowanie ruchu**: trapezoidalny profil prędkości dla płynnych ruchów

7.2 System Sterowania

- **Regulacja PID**: dla każdej osi (kontrola dokładności)
- **Kalibracja**: automatyczna kalibracja przy starcie
- **Filtrowanie sygnałów**: dla płynnego ruchu

7.3 Interfejs Użytkownika

Aplikacja desktopowa (Python/C++) z następującymi funkcjami i komponentami:

- Kalibracja każdego potencjometru osobno
- Ustawienie dokładności i szybkości ruchu
- Wyświetlanie pozycji w czasie rzeczywistym
- Rejestrowanie i odtwarzanie ruchów

8. Schematy Podłączeń

8.1 Schemat Główny Systemu

8.2 Podłączenia Embedded

```
MASTER ARDUINO MEGA 2560
  D20 ──── I²C SDA
  D21 ──── I²C SCL
  TX1 ──── UART TX (komunikacja)
  RX1 ──── UART RX (komunikacja)

A0 ──── POT. OŚ 1
A1 ──── POT. OŚ 2
D2 ──── TTP229B OŚ 1
D3 ──── TTP229B OŚ 2
D4 ──── MICRO SWITCH STOP
D5 ──── MICRO SWITCH EMERGENCY
D6 ──── MICRO SWITCH START
D7 ──── MICRO SWITCH RESET
```

8.3 Podłączenia Sterowniczo/Sprzętowe

```
MASTER ARDUINO MEGA 2560

PWM 1 ──── SERWO OŚ 1
PWM 2 ──── SERWO OŚ 2
PWM 3 ──── SERWO OŚ 3
PWM 4 ──── SERWO OŚ 4
PWM 5 ──── SERWO OŚ 5
PWM 6 ──── SERWO OŚ 6

STEP 1 ──── NEMA 17 OŚ 1
DIR 1 ──── NEMA 17 OŚ 1
STEP 2 ──── NEMA 17 OŚ 2
DIR 2 ──── NEMA 17 OŚ 2

SLAVE ARDUINO MEGA 2560 / ESP32

USB ──── KAMERA USB HD 1080p

ZASILACZ 12V 10A ──── SERWOMECHANIZMY
ZASILACZ 5V 3A ──── STEROWNIKI, SENSORY
```

9. Instrukcja Montażu

9.1 Przygotowanie Komponentów

Części Mechaniczne

- **Elementy Drukowane 3D**: zamówić jakość wydruku
- **Śruby i Nakrętki**: M3, M4, M5 – sprawdzić długości
- **Łożyska**: 6x 608ZZ, 8x 6000ZZ – sprawdzić luz
- **Serwomechanizmy**: 6x MG996R, 2x NEMA 17 – testować przed montażem
- **Narzędzia**: śrubokręt, imbusy, multimetr, lutownica
- **Taśmy dwustronne**: do mocowania elementów

Części Elektroniczne

- **Płytka Stykowa**: 830 punktów
- **Przewody**: Dupont M-M, M-F, F-F, 22 AWG
- **Arduino Mega 2560**: 2x (Master, Slave)
- **Kamera USB HD**: 1x z autofocusem
- **Potencjometry**: 10 kΩ, 2x
- **Przyciski**: 4x (Stop, Emergency, Start, Reset)
- **Zasilacze**: 12V 10A, 5V 3A
- **Przewody i złącza**: JST-XH, przewody zasilające

9.2 Montaż Mechaniczny

Przygotowanie podstawy

1. Przymocować podstawę do stołu roboczego za pomocą śrubami
2. Zainstalować kolumnę centralną
3. Sprawdzić czy kolumna jest pionowa

Montaż ramienia bazowego

1. Przymocować serwo obrotowe do podstawy
2. Zamontować ramię bazowe
3. Sprawdzić zakres ruchu

Montaż przedramienia

1. Przymocować serwo do ramienia bazowego
2. Zamontować przedramię
3. Sprawdzić zakres ruchu

Montaż nadgarstka i chwytaka

1. Przymocować serwo nadgarstka
2. Zamontować chwytak
3. Sprawdzić zakres ruchu

9.3 Montaż Elektroniczny

Przygotowanie płytek

1. Arduino Master (Sterownik) – zamontować na płytce prototypowej
2. Arduino Slave (Wizualizacja) – zamontować PO KAMERĄ
3. Zasilacze – zamontować

Połączenia

1. Podłączyć 6 serwomechanizmów do PCA9685
2. Podłączyć 2 silniki krokowe do PCA9685
3. Podłączyć 6 potencjometrów do ADC
4. Podłączyć 4 przyciski do GPIO
5. Podłączyć PCA9685 przez I²C
6. Podłączyć I²C pomiędzy Master i Slave

9.4 Montaż Mechaniczny

Przygotowanie podstawy

1. Przymocować podstawę do stołu roboczego za pomocą przykręcanych
2. Zainstalować kolumnę centralną
3. Sprawdzić czy kolumna jest pionowa

Montaż ramienia bazowego

1. Przymocować serwo obrotowe do podstawy
2. Zamontować ramię bazowe
3. Sprawdzić zakres ruchu

Montaż przedramienia

1. Przymocować serwo do ramienia bazowego
2. Zamontować przedramię
3. Sprawdzić zakres ruchu

Montaż nadgarstka i chwytaka

1. Przymocować serwo nadgarstka
2. Zamontować chwytak
3. Sprawdzić zakres ruchu

9.5 Montaż Elektroniczny

Przygotowanie płytek

1. Arduino Master (Sterownik) – zamontować na płytce prototypowej
2. Arduino Slave (Wizualizacja) – zamontować PO KAMERĄ
3. Zasilacze – zamontować

Połączenia

1. Podłączyć 6 serwomechanizmów do PCA9685
2. Podłączyć 2 silniki krokowe do PCA9685
3. Podłączyć 6 potencjometrów do ADC
4. Podłączyć 4 przyciski do GPIO
5. Podłączyć PCA9685 przez I²C
6. Podłączyć I²C pomiędzy Master i Slave

10. Konserwacja

Częstotliwość	Zadanie	Opis
Codziennie	Sprawdzenie, czy mechanizm działa	Sprawdzić czy wszystkie serwa się poruszają płynnie
Tygodniowo	Przed kalibracją systemu	Wymiana potencjometrów wymagana
Miesięcznie	Wymiana łożysk	Wymiana potencjometrów wymagana

11. Rozwiązywanie Problemów

11.1 System Problemów

Problem	Przyczyna	Rozwiązanie
Drgania	Luz kinetyk, niewłaściwe kalibracja	Sprawdzić kinetyk, wymienić, kalibracja, kalibracja
Brak komunikacji	Kabel I²C, zasilanie, adres	Sprawdzić połączenia, zasilanie, adres, adres
Brak komunikacji	Kabel I²C, zasilanie, adres	Sprawdzić połączenia, zasilanie, adres, adres
Przegrzewanie serwo	Przeciążenie, złe ustawienie	Sprawdzić obciążenie, sprawdzić ustawienie

Ostrzeżenie: Nigdy nie pracuj przy włączonym zasilaniu bez odłączonego kabla!

11.2 Diagnostyka

- Sprawdzić logi systemu, czy zostały zbierane diagnostyczne w serwomechanizmy
- Monitorować, sprawdzić temperaturę, przy przeciążeniu (wyjątek)
- Logi: Analiza logów błędów (gdy wdrożone)

12. Bezpieczeństwo

12.1 Zabezpieczenia

- Emergency Stop: Przycisk awaryjnego zatrzymania
- Limity: Ograniczenie zakresu ruchu (software)
- Monitoring: Czujki temperaturowe serwo (opcjonalnie)

12.2 Procedury Bezpieczeństwa

1. Przed pracą: Sprawdź wszystkie połączenia
2. Podczas pracy: Trzymaj ręce/włosy z dala od
3. Po pracy: Wyłącz zasilanie, odłącz kabel

Ostrzeżenie: Nigdy nie pracuj przy włączonym zasilaniu bez odłączonego kabla!

13. Kod Źródłowy

13.1 Arduino Master (Kinematyka, Kontrola)

Zawiera: Kod master, parsing, kinematyka, komunikacja I²C, Master Slave, przetwarzanie ADC, Arduino Mega 2560 (Sterownik Główny)

```
// Master Sterownik - Arduino Mega 2560 (Sterownik Główny)
// Wersja 1.0
// Autor: Adam Wolf

// Biblioteki
#include <Wire.h>
#include <Servo.h>
#include <AccelStepper.h>
#include <MultiStepper.h>

// Definicje pinów potencjometrów
#define POT_OS1 A0
#define POT_OS2 A1
#define POT_OS3 A2
#define POT_OS4 A3
#define POT_OS5 A4
#define POT_OS6 A5

// Definicje pinów przycisków
#define BUTTON_STOP 2
#define BUTTON_EMERGENCY 3
#define BUTTON_START 4
#define BUTTON_RESET 5

// Definicje pinów serwomechanizmów
#define SERVO_OS1 6
#define SERVO_OS2 7
#define SERVO_OS3 8
#define SERVO_OS4 9
#define SERVO_OS5 10
#define SERVO_OS6 11

// Definicje pinów silników krokowych
#define STEP_OS1 12
#define DIR_OS1 13
#define STEP_OS2 14
#define DIR_OS2 15

// Adres I²C dla komunikacji z Slave
#define SLAVE_ADDRESS 0x08

// Obiekty serwomechanizmów
Servo servo1, servo2, servo3, servo4, servo5, servo6;

// Obiekty silników krokowych
AccelStepper stepper1(AccelStepper::DRIVER, STEP_OS1, DIR_OS1);
AccelStepper stepper2(AccelStepper::DRIVER, STEP_OS2, DIR_OS2);

// Zmienne globalne
int pot_values[6];
int servo_positions[6];
bool emergency_stop = false;

// Struktura danych do wysłania do Slave
struct __attribute__((packed)) SlaveData {
  int16_t pot_values[6];
  int16_t servo_positions[6];
  uint8_t button_states;
  uint8_t checksum;
};

// Struktura danych do odebrania od Slave
struct __attribute__((packed)) SlaveCommand {
  uint8_t command_type;
  int16_t target_positions[6];
  uint8_t checksum;
};

// Funkcja obliczająca sumę kontrolną
uint8_t calculate_checksum(uint8_t* data, size_t length) {
  uint8_t sum = 0;
  for (size_t i = 0; i < length; i++) {
    sum += data[i];
  }
  return sum;
}

// Funkcja wysyłająca dane do Slave
void send_to_slave() {
  SlaveData data;
  for (int i = 0; i < 6; i++) {
    data.pot_values[i] = pot_values[i];
    data.servo_positions[i] = servo_positions[i];
  }
  data.button_states = 0;
  data.checksum = calculate_checksum((uint8_t*)&data, sizeof(data) - 1);

  Wire.beginTransmission(SLAVE_ADDRESS);
  Wire.write((uint8_t*)&data, sizeof(data));
  Wire.endTransmission();
}

// Funkcja odbierająca dane od Slave
void receive_from_slave() {
  Wire.requestFrom(SLAVE_ADDRESS, sizeof(SlaveCommand));
  if (Wire.available() >= sizeof(SlaveCommand)) {
    SlaveCommand cmd;
    Wire.readBytes((uint8_t*)&cmd, sizeof(cmd));

    uint8_t checksum = calculate_checksum((uint8_t*)&cmd, sizeof(cmd) - 1);
    if (checksum == cmd.checksum) {
      // Przetwarzanie komendy
      if (cmd.command_type == 1) {
        for (int i = 0; i < 6; i++) {
          servo_positions[i] = cmd.target_positions[i];
        }
      }
    }
  }
}

// Funkcja odczytująca potencjometry
void read_potentiometers() {
  pot_values[0] = analogRead(POT_OS1);
  pot_values[1] = analogRead(POT_OS2);
  pot_values[2] = analogRead(POT_OS3);
  pot_values[3] = analogRead(POT_OS4);
  pot_values[4] = analogRead(POT_OS5);
  pot_values[5] = analogRead(POT_OS6);
}

// Funkcja sterująca serwomechanizmami
void update_servos() {
  servo1.write(servo_positions[0]);
  servo2.write(servo_positions[1]);
  servo3.write(servo_positions[2]);
  servo4.write(servo_positions[3]);
  servo5.write(servo_positions[4]);
  servo6.write(servo_positions[5]);
}

// Funkcja obsługująca przyciski
void handle_buttons() {
  if (digitalRead(BUTTON_EMERGENCY) == LOW) {
    emergency_stop = true;
    stepper1.stop();
    stepper2.stop();
  }
  if (digitalRead(BUTTON_RESET) == LOW) {
    emergency_stop = false;
  }
}

// Setup
void setup() {
  Serial.begin(115200);
  Wire.begin();

  // Inicjalizacja serwomechanizmów
  servo1.attach(SERVO_OS1);
  servo2.attach(SERVO_OS2);
  servo3.attach(SERVO_OS3);
  servo4.attach(SERVO_OS4);
  servo5.attach(SERVO_OS5);
  servo6.attach(SERVO_OS6);

  // Inicjalizacja silników krokowych
  stepper1.setMaxSpeed(1000);
  stepper1.setAcceleration(500);
  stepper2.setMaxSpeed(1000);
  stepper2.setAcceleration(500);

  // Inicjalizacja przycisków
  pinMode(BUTTON_STOP, INPUT_PULLUP);
  pinMode(BUTTON_EMERGENCY, INPUT_PULLUP);
  pinMode(BUTTON_START, INPUT_PULLUP);
  pinMode(BUTTON_RESET, INPUT_PULLUP);

  Serial.println("Master Sterownik - Gotowy");
}

// Loop
void loop() {
  read_potentiometers();
  handle_buttons();

  if (!emergency_stop) {
    update_servos();
    send_to_slave();
    receive_from_slave();
  }

  delay(10);
}
```

13.2 Arduino Slave (Wizualizacja)

Zawiera: Kod slave, parsing, komunikacja, wizualizacja, komunikacja przez PC/UART, Arduino Mega 2560 (Sterownik Wizyjny)

```
// Slave Sterownik - Arduino Mega 2560 (Sterownik Wizyjny)
// Wersja 1.0
// Autor: Adam Wolf

// Biblioteki
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// Definicje pinów wyświetlacza
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
#define SCREEN_ADDRESS 0x3C

// Obiekt wyświetlacza
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// Adres I²C dla komunikacji z Master
#define SLAVE_ADDRESS 0x08

// Struktura danych do odebrania od Master
struct __attribute__((packed)) SlaveData {
  int16_t pot_values[6];
  int16_t servo_positions[6];
  uint8_t button_states;
  uint8_t checksum;
};

// Struktura danych do wysłania do Master
struct __attribute__((packed)) SlaveCommand {
  uint8_t command_type;
  int16_t target_positions[6];
  uint8_t checksum;
};

// Zmienne globalne
SlaveData received_data;
bool data_received = false;

// Funkcja obliczająca sumę kontrolną
uint8_t calculate_checksum(uint8_t* data, size_t length) {
  uint8_t sum = 0;
  for (size_t i = 0; i < length; i++) {
    sum += data[i];
  }
  return sum;
}

// Funkcja odbierająca dane od Master
void receive_from_master() {
  Wire.requestFrom(SLAVE_ADDRESS, sizeof(SlaveData));
  if (Wire.available() >= sizeof(SlaveData)) {
    Wire.readBytes((uint8_t*)&received_data, sizeof(received_data));

    uint8_t checksum = calculate_checksum((uint8_t*)&received_data, sizeof(received_data) - 1);
    if (checksum == received_data.checksum) {
      data_received = true;
    }
  }
}

// Funkcja wysyłająca komendę do Master
void send_to_master(uint8_t command_type, int16_t* target_positions) {
  SlaveCommand cmd;
  cmd.command_type = command_type;
  for (int i = 0; i < 6; i++) {
    cmd.target_positions[i] = target_positions[i];
  }
  cmd.checksum = calculate_checksum((uint8_t*)&cmd, sizeof(cmd) - 1);

  Wire.beginTransmission(SLAVE_ADDRESS);
  Wire.write((uint8_t*)&cmd, sizeof(cmd));
  Wire.endTransmission();
}

// Funkcja wyświetlająca dane na ekranie
void display_data() {
  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0, 0);

  display.println("Slave Sterownik");
  display.println("Wizualizacja");
  display.println("");

  if (data_received) {
    display.println("Potencjometry:");
    for (int i = 0; i < 6; i++) {
      display.print("OS");
      display.print(i + 1);
      display.print(": ");
      display.println(received_data.pot_values[i]);
    }
  } else {
    display.println("Brak danych");
  }

  display.display();
}

// Setup
void setup() {
  Serial.begin(115200);
  Wire.begin();

  // Inicjalizacja wyświetlacza
  if (!display.begin(SSD1306_SWITCHCAP, SCREEN_ADDRESS)) {
    Serial.println(F("SSD1306 allocation failed"));
    for (;;);
  }

  display.clearDisplay();
  display.display();

  Serial.println("Slave Sterownik - Gotowy");
}

// Loop
void loop() {
  receive_from_master();
  display_data();

  delay(100);
}
```

14. Lista Komponentów

14.1 Mikrokontrolery

- 2x Arduino Mega 2560 / Arduino Nano (lub 2x ESP32 Dev Kit C)
- 1x Płytka prototypowa

14.2 Serwomechanizmy i Aktuatory

- 6x PCA9685 – Sterowniki serwomechanizmów PWM/I2C
- 6x MG996R – Serwomechanizmy (główne osie)
- 4x TTP229B – Czujniki dotyku pojemnościowego
- 4x Micro Switch – Przyciski

14.3 Serwomechanizmy

- 6x MG996R – Serwomechanizmy (długość ramion: przedramię, ramię, przedramię, nadgarstek, chwyt)
- 2x SG90 – Serwomechanizmy (obrót podstawy)

14.4 Zasilanie

- 1x Zasilacz 5V 3A – Elektronika
- 1x Zasilacz 12V 5A – LED Ring

14.5 Okablowanie

- Przewody połączeniowe 1x Dupont (długość 30cm)
- Przewody zasilające 1x Dupont (długość 30cm)
- Taśma, 5V, 1 kolorowa (do ramion)
- Taśma dwustronna (do montażu)
- Taśma izolacyjna (do izolacji)

Dokument wygenerowano: 25 października 2025, 20:00:00
Autor: Adam Wolf
Projekt: System Teleoperacyjny do Precyzyjnych Prac Elektronicznych
Wersja: Kompletna Dokumentacja Techniczna

Dokumentacja techniczna przygotowana na podstawie specyfikacji technicznej i kodu źródłowego dostarczonego przez użytkownika.